

LLMs Meet Post-Silicon Test Engineering: A New Era (Invited)

Li-C. Wang

University of California, Santa Barbara
licwang@ece.ucsb.edu

Abstract—The transformative power of Large Language Models (LLMs) is reshaping the role of AI in post-silicon test engineering. This paper summarizes our experience in leveraging LLMs to develop AI agents specifically tailored for this domain. Central to our approach is a two-stage process: first, we utilize the reasoning capabilities of LLMs to systematically interpret user queries; second, we invoke a grounding process to execute tasks as directed by these queries. This grounding ensures seamless integration of the LLM with existing test engineering infrastructure, enabling the AI agent to autonomously perform tasks within an established framework. Using the Intelligent Engineering Assistant (IEA) as a case study, we demonstrate how domain-specific, LLM-powered AI agents can automate critical aspects of test engineering, and highlight the potential of LLMs to revolutionize post-silicon test engineering through intelligent, context-aware automation.

Index Terms—Large Language Models, Semiconductor Testing

I. INTRODUCTION

The concept of the Intelligent Engineering Assistant (IEA) was first introduced in 2018 [1], initially driven by the fundamental question: “What does applied machine learning mean in test data analytics?” At that time, machine learning (ML) had already been a prominent topic within test engineering for years. The prevailing view of “applied ML” emphasized the need for developing novel ML tools or models specifically tailored to semiconductor testing.

The work in [2] cast doubts about this perspective. The work was about solving a yield problem that significantly impacted an automotive SoC product line in high-volume production. Our effort concentrated on identifying an E-Test parameter (i.e., a PCM—process control monitor—parameter) exhibiting a high correlation with the yield loss. Once such E-Test parameters were identified, we could request the foundry to adjust the manufacturing process accordingly, thereby improving the yield. During this effort, we discovered five such parameters, leading to two process adjustments whose combined effects significantly enhanced yield, as verified by silicon results [2].

While the result was promising, the work clearly demonstrated that having the necessary tools or models to conduct the analysis was only part of the solution. The success largely depended on knowing how to apply these tools or models effectively and interpret their outcomes properly. Figure 1 illustrates this aspect. Our analysis involved an iterative search process, where each step focused on analyzing an extracted dataset D , using a particular tool T , guided by a query Q in mind. Results from each step were summarized visually, enabling us to evaluate the findings and determine subsequent steps.

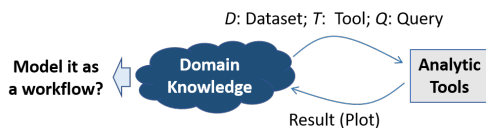


Fig. 1. IEA2018 intended to model domain knowledge as a workflow

In addition to the yield optimization task [2], the knowledge-driven search perspective was also evident in our other works, as discussed in [3]. This perspective suggested that incorporating domain-specific knowledge was essential for building automated solutions capable of replicating successes from one scenario to future instances (e.g., addressing similar problems). In an initial attempt [4], domain knowledge derived from our yield optimization effort was captured using a *process model*, and the resulting solution was successfully applied to a second product line. Additionally, this process modeling approach was employed in the context of design verification [5] to facilitate functional test generation. At that stage, the core idea behind modeling domain knowledge involved first defining a set of *primitives* and then representing each analysis step as a sequence of these primitives. Although this approach proved effective for the specific scenarios in those studies, its generalization remained unclear.

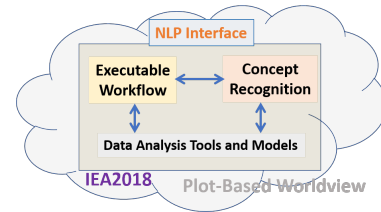


Fig. 2. IEA2018 introduced the idea of concept-driven workflow programming

The introduction of IEA in 2018, referred to as IEA2018, aimed to move beyond primitive-based knowledge modeling by representing domain knowledge as an *executable workflow*. Each step in this workflow addressed an existence question about a specific *concept*. For instance, concepts such as *high correlation* or *low yield* could be explicitly defined. Subsequently, data analysis tools or models were employed to determine whether these concepts existed within a given dataset. This methodology resulted in the three-component design of IEA2018, illustrated in Figure 2. Within the IEA2018 framework, applied ML was conceptualized as developing ML tools or models capable of addressing decision problems formulated as: “Does concept C exist in the dataset?” Note that multiple decision procedures may be associated with a concept to determine its presence, with the specific choice depending on the context [6].

IEA2018 was designed as an autonomous system [1], operating within a *plot-based worldview* that specified various plot types to prescribe its outputs. The system accepted user queries formulated as analytical questions, such as “Do you see a yield issue in the last two weeks?” If a yield issue was identified, it would trigger additional analyses following a defined yield analytic workflow—for instance, correlating yield loss with E-test data. The natural language processing (NLP) frontend of IEA2018 was implemented using NLP platforms available at that time, such as [7]. As recently discussed in [6], IEA2018 emphasized developing standardized workflows to

capture engineering practices within specific application areas, such as yield optimization or test cost reduction. This standardization encompassed three levels: defining a standardized set of concepts, standardizing the procedural steps (or *instructions*) that workflows could invoke, and standardizing the workflows themselves. Consequently, deploying an IEA required creating a *complete* specification of a team’s engineering practices within a company—ultimately becoming a major obstacle for the practical deployment of IEA2018.

II. CO-EVOLUTION OF IEA AND LLMs

The availability of the GPT-3 model [8] inspired new thinking regarding IEA. Although the central innovation of GPT-3 was its ability for *in-context learning*, it was actually GPT-3’s strong paraphrasing capabilities that caught our attention. These capabilities enabled the feasible implementation of a robust *semantic parser*. *Semantic parsing* refers to the task of assigning real-world meanings to linguistic inputs, such as words or sentences. Traditionally, semantic parsing approaches relied on supervised learning methods, requiring substantial training data [9]. Such an approach was not favored in our domain due to the high effort required to produce sufficient training data. An alternative method, known as *semantic parsing via paraphrasing* [10], emerged as a more practical solution. Leveraging GPT-3’s paraphrasing strengths, the feasibility of implementing *constrained semantic parsing* was demonstrated [11].

IEA2022 [12][13] thus departed from the idea of achieving an autonomous system. Instead, IEA2022 implemented a semantic parser capable of converting user instructions in English into *calling templates*, which then invoke API functions in the backend. Figure 3 illustrates this new design approach. The development of IEA2022 involved organizing analytic tools to provide an API interface, defining rules to construct calling templates, and leveraging GPT-3 to implement the semantic parser. For semantic parsing, the calling templates in IEA2022 corresponded directly to the *meaning representations* introduced in [11].

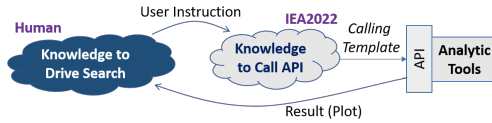


Fig. 3. IEA2022 separated the domain knowledge into two parts

As depicted in Figure 3, IEA2022 effectively separated the domain knowledge into two parts: the knowledge to drive the search and the knowledge to execute the search, e.g. calling backend API. The first part of the domain knowledge was left to the user and consequently, IEA2022 did not dictate how users should conduct an analytic flow. It should be noted that recently LLMs have significantly expanded their capabilities beyond mere text generation, enabling sophisticated interactions with external tools through dedicated *tool-calling* mechanisms [14] [15] [16] [17] (see also LangChain [18]). In retrospect, the main purpose of IEA2022 was to achieve *robust* tool-calling capabilities in view of the domain-specific backend API.

Robustness was a key consideration for IEA2022, as the system explicitly prescribed the set of *acceptable* instructions. This prescription was accomplished by defining a set of *canonical utterances* [11], such that only these canonical utterances and their paraphrases were considered acceptable inputs. For each acceptable instruction, the mapping to its corresponding calling template was precisely defined, eliminating ambiguity—IEA2022 traded flexibility for robustness.

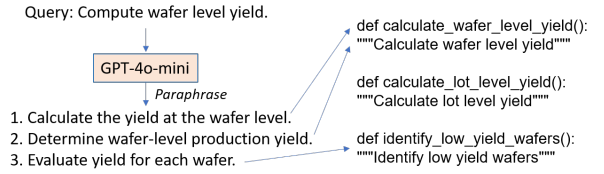


Fig. 4. Automatic tool calling can depend on semantic interpretation

The tool-calling capabilities supported by recent LLMs suggest that IEA2022 could potentially be realized with a much simpler implementation using an agentic programming software platform such as LangChain [18]. With tool calling, we could bypass many detailed aspects of semantic parsing. However, this does not eliminate all potential challenges. For instance, Figure 4 presents a simple example illustrating that the effectiveness of tool calling still largely depends on the semantic interpretation between the user’s query and the descriptive documentation associated with a given function. In this example, the query “Compute wafer level yield” was paraphrased by GPT-4o-mini into three different statements, and a simple agent implemented using the LangChain platform was tasked with invoking the corresponding functions. While the first two paraphrases correctly resulted in calling the intended function, the third paraphrase unexpectedly triggered a different function.

This ambiguity arose because the paraphrase “Evaluate yield for each wafer” could reasonably imply either “Calculate wafer level yield” or “Identify low yield wafers,” depending on the semantic interpretation of the verb “Evaluate.” Additionally, as noted in the LangChain documentation [18], simple and narrowly-scoped tools are generally easier to manage than complex ones, and selecting from a long list of available tools remains a challenging aspect. In summary, IEA2022 and contemporary tool-calling mechanisms can be viewed as two alternatives: one prioritizing robustness, and the other emphasizing flexibility and ease of implementation.

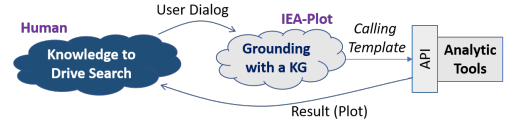


Fig. 5. IEA-Plot in 2023 formulates a grounding problem

With the availability of GPT-3.5 Turbo [19], IEA2022 evolved into IEA-Plot in 2023 [20]. Unlike IEA2022, which treated the mapping from user instructions to calling templates as a semantic parsing problem, IEA-Plot approached this mapping as solving a *task grounding problem* [21][22]. Specifically, IEA-Plot introduced a *knowledge graph* (KG) within its architecture and utilized this KG to guide the grounding process [6], as illustrated in Figure 5.

As explained in [6], IEA-Plot’s task grounding involves identifying relevant terms and phrases in a user query and linking these linguistic elements to corresponding nodes within the KG. This linkage enables the extraction of a subgraph that captures the semantic meaning of the query. The extracted subgraph may also include default nodes to supply any necessary information not explicitly mentioned by the user. For instance, the query “Do you see systematic wafer map patterns?” would link the verb “see” to an action node representing wafer map pattern analysis. Similarly, the object phrase “systematic wafer map patterns” would be linked to the concept node “systematic patterns,” applying a default meaning for the term “systematic.”

Furthermore, the query misses a time range, triggering the inclusion of a default range (e.g., the entire dataset), or reuse of the time range mentioned in a previous query, if it existed.

The use of a knowledge graph (KG) enabled IEA-Plot to maintain a set of *options* for each action-object pair, allowing it to carry user-specified options from one query to the next. Consider, for instance, the two queries: "What is the yield trend in the last two weeks?" and "Do you see a yield excursion?" Here, the specified time range ("last two weeks") from the first query could be automatically propagated to fill in the missing time range option in the subsequent query. The KG in IEA-Plot predefined these sets of *options* for each action-object pair and facilitated their propagation between queries. Thus, unlike IEA2022, which treated each instruction independently, IEA-Plot provided greater flexibility to support execution of dialogs—sequences of potentially interconnected instructions.

III. DEPLOYMENT OBSTACLES FACED BY IEA-PLOT

After the release of IEA-Plot, we spent a year implementing it within an industrial setting. This deployment effort revealed two primary areas of challenges: acquiring well-defined user requirements and ensuring scalability as new requirements emerged.

Although IEA-Plot eliminated the necessity of standardized workflows, it still required a comprehensive specification of API functions to support user tasks. Compared to IEA2022, IEA-Plot broadened the range of acceptable instructions and supported sequences of instructions (dialogs). However, users were not free to enter arbitrary instructions; if a requested task was unsupported by existing API functions, new functions had to be developed. This tight coupling between user instructions and the backend API significantly limited the ease and speed of expanding IEA-Plot's capabilities.

A more fundamental challenge involved the acquisition of user requirements. Initially, our strategy for deploying IEA-Plot depended heavily on gathering detailed requirements from users through meetings, aiming for a comprehensive API specification. This approach quickly proved problematic. Engineers and managers typically provided only high-level descriptions or limited examples of their intended tasks, rather than the extensive, detailed instruction lists expected by IEA-Plot. Expecting teams to thoroughly document existing tasks and predict their future needs was often impractical. Additionally, many user-provided examples were inherently complex, involving multi-step scenarios beyond the simpler, single-goal instructions originally envisioned by the IEA-Plot design.

IV. LATEST ADVANCEMENTS IN LLMs

Recent advancements in LLMs have significantly improved their capacity to handle complex, multi-step reasoning tasks. Techniques such as Chain-of-Thought (CoT) prompting [23] have made intermediate reasoning steps more transparent within these models. As summarized in a recent survey [24], advances in LLM reasoning broadly encompass enhanced prompting strategies, architectural improvements, and novel training paradigms. OpenAI's GPT-o3-mini [25], released in January 2025, exemplifies these advancements by providing a compact yet powerful model optimized specifically for reasoning tasks. The introduction of GPT-o3-mini and its more capable variant, GPT-o3-mini-high, presented new opportunities to overcome the limitations identified in the IEA-Plot design.

In parallel, the emergence of agentic programming platforms such as LangGraph [26], which became stable in June 2024, provided tools for constructing modular and flexible AI applications using computational graphs. LangGraph simplifies the creation of sophisticated AI agents by supporting structured interactions among multiple

LLMs, external tools, and conditional logic. Drawing inspiration from recent AI-agent methodologies such as ReAct [27] and Reflexion [28], LangGraph has significantly streamlined the development of interactive, reasoning-capable LLM-powered AI agents.

Motivated by these developments, we revisited our design of IEA once again. By leveraging GPT-4o, GPT-o3-mini, GPT-o3-mini-high, and the LangGraph platform, we developed a new IEA component called IEA-Plugin—an agentic framework designed for workflow reasoning and automated API generation within our IEA family.

V. IEA-PLUGIN

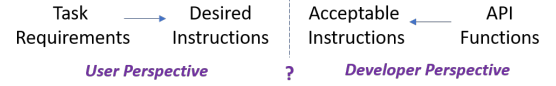


Fig. 6. The challenges lie in resolving the two perspectives

The challenges discussed in Section III are summarized in Figure 6, highlighting the dilemma between two contrasting perspectives. From the viewpoint of IEA developers, it's preferable to define a limited set of acceptable instructions and ensure all user inputs align with this predefined scope. Conversely, from the users' perspective, practical task requirements naturally dictate necessary instructions, which may extend beyond this scope. Given the dynamic nature of real-world test engineering tasks, it is generally impractical to restrict users strictly to a predetermined set of instructions.

Traditional thinking to resolve this dilemma involved seeking a standard set of instructions that accommodated both sides as closely as possible. However, recent advances in code-generation capabilities of LLMs had led us to reconsider this strategy. Previously, developers limited the set of acceptable instructions primarily to avoid frequent and extensive modifications to the backend APIs. If such programming efforts could be significantly reduced, the developer's constraint became less critical. Consequently, the next generation of IEA design could more directly emphasize the user's perspective.

Following this thinking, we developed the IEA-Plugin [29] to address the challenges discussed in Section III. Figure 7 illustrates the main components and typical usage of IEA-Plugin.

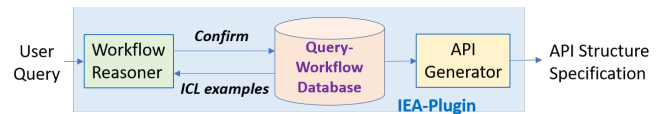


Fig. 7. IEA-Plugin acquires user queries and generates an API structure

IEA-Plugin is designed as an interactive tool for users. When a user enters a query, its workflow reasoner generates a workflow consisting of a sequence of *steps*, as illustrated by the example in Figure 8. Once the workflow is reviewed and confirmed, the query-workflow pair is stored in a database, which accumulates all user interactions and generated workflows. The workflow reasoner in IEA-Plugin is powered by GPT-o3-mini. For each new query, the *in-context learning* (ICL) examples included in the prompt are selected from the existing query-workflow pairs stored in this database (see [29]).

The reasoner in IEA-Plugin was implemented as a LangGraph agent, as illustrated in Figure 9. This agent combined key concepts from both Retrieval-Augmented Generation (RAG) and ReAct [27] agent designs. Inspired by the Chain-of-Thought (CoT) prompting method [23], the workflow generation prompt provided to GPT-o3-mini explicitly instructs the model to generate the *thought* behind

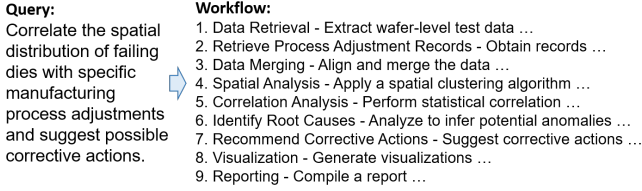


Fig. 8. An example to illustrate workflow generation in IEA-Plugin

the generated workflow. Upon receiving a query, the agent initially retrieves the semantically most similar existing queries from the database by comparing semantic embedding vectors—an approach commonly used in RAG. These retrieved queries serve as the ICL examples within the prompt in the first run.

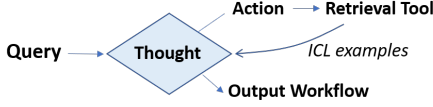


Fig. 9. IEA-Plugin's reasoner leveraging thought and retrieval action

After prompting GPT-o3-mini, the reasoner produces a workflow along with the associated thought. In subsequent iterations, the generated thought is used to perform the similarity search, this time focusing on comparing thoughts rather than comparing queries. This thought-based retrieval approach ensures that ICL examples reflect reasoning processes closely aligned with the query at hand. The deliberation process continues iteratively until the similarity score between two consecutively generated thoughts surpasses a predefined threshold (e.g., 0.9). At that point, the agent concludes deliberation and outputs the workflow corresponding to the last thought.

The workflow reasoner serves as a user requirements acquisition tool. Rather than requiring teams to document their requirements through detailed instructions, users can simply interact with IEA-Plugin by submitting queries. Utilizing the reasoning capabilities of GPT-o3-mini, IEA-Plugin automatically accumulates workflow knowledge based on these interactions. Additionally, by leveraging the generalization capabilities of GPT-4o, IEA-Plugin includes a query generator capable of dramatically expanding the scope of existing queries within a given application context, such as yield analysis (see [29]). Consequently, users only need to provide a few query examples, enabling IEA-Plugin to generate an extensive and comprehensive set of queries (and their workflows).

As shown in Figure 7, IEA-Plugin includes an API generator, which leverages the coding capabilities of the GPT-o3-mini-high model. First, the query examples stored in the database undergo classification using GPT-4o. Subsequently, each group of classified instances—consisting of query-thought-workflow triples—is provided to GPT-o3-mini-high, which generates an API specification covering all the *steps* appearing in the workflows. The resulting specification prescribes an API structure, listing each function and its covered steps, along with documentation. No actual code to implement a function is produced at this stage. Figure 10 shows an example of the generated API structure (for further details, see [29]).

While the reasoner effectively addresses the requirements acquisition challenge discussed in Section III, the API generator specifically resolves the scalability issue. Through experiments, we observed that the combined process of classification and API generation consistently resulted in a highly systematic API structure. Although

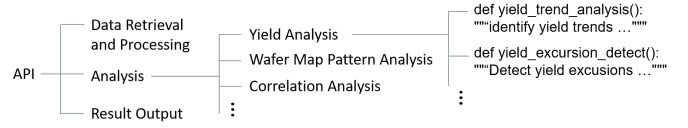


Fig. 10. Illustration of an API structure specification

new query-workflow pairs could introduce additional functions, the overall API structure remained largely *stable*. In retrospect, achieving a stable API structure was one of the most challenging aspects for scaling the functionalities in IEA-Plot. LLMs addressed this concern by enabling the automatic generation of a coherent and stable API structure. Once the API structure is stable, today's LLM coding capabilities can significantly reduce the effort required to implement individual, well-defined functions.

VI. LESSONS LEARNED

Having worked in the semiconductor testing field for years, we were accustomed to prioritizing concerns such as quality and robustness. From this viewpoint, the inherently statistical nature of LLMs initially appeared disadvantageous. However, throughout the evolution of the IEA framework, we discovered that while robustness remains critical from one perspective, the variability of LLM outputs could be beneficial from another (see Figure 6). Specifically, variability can be interpreted positively as a form of "creativity." For example, multiple workflow alternatives can be generated, allowing users to select the most suitable option. Rather than directly asking LLMs to produce implementation code—which could introduce errors—we leveraged them primarily to generate structured API specifications with detailed documentation to guide human developers. While the current IEA framework still incorporates human-in-the-loop steps, such as reviewing and confirming workflows or revising generated code, these activities no longer represent significant bottlenecks compared to previous IEA designs.

From a domain-specific standpoint, one of our early concerns was whether LLMs possessed adequate knowledge within our specialized field of semiconductor testing. Recent research has highlighted the growing interest and challenges in assessing the knowledge content and boundaries of LLMs [30][31][32][33][34]. Through our experience developing IEA-Plugin, we discovered that contemporary LLMs indeed contain substantial domain-specific knowledge on test engineering, as demonstrated by the quality and comprehensiveness of the generated workflows and API specifications.

In refining the prompts used by IEA-Plugin, we initially grappled with common concerns such as self-consistency [35][36], prompt sensitivity [37][38][39], and ICL sensitivity [40][41][42]. After extensive experimentation, we adopted an ICL-centric strategy (see [29]), carefully selecting consistent and relevant examples [43], while keeping task instructions concise and intuitive. Additionally, techniques such as CoT prompting [23] and ReAct [27] proved particularly effective, consistently yielding higher-quality outcomes. Regarding self-consistency, our experiences showed that the first response typically represented the most "nominal" or probable outcome, often aligning closely with intuitive expectations. Ultimately, by integrating GPT-4o, GPT-o3-mini, and GPT-o3-mini-high into LangGraph agents, we successfully implemented IEA-Plugin without extensive manual coding. This highlights a fundamental shift from earlier IEA designs, demonstrating that modern LLM capabilities significantly streamline the development of domain-specific AI assistants.

REFERENCES

- [1] L.-C. Wang, "An autonomous system view to apply machine learning," in *IEEE International Test Conference*, 2018.
- [2] J. Tikkanen, S. Siatkowski, N. Sumikawa, L.-C. Wang, and M. S. Abadir, "Yield optimization using advanced statistical correlation methods," *IEEE International Test Conference*, 2014.
- [3] L.-C. Wang, "Experience of data analytics in EDA and test - principles, promises, and challenges," *IEEE Transactions on CAD*, vol. 36, no. 6, pp. 885–898, 2017.
- [4] S. Siatkowski, L.-C. Wang, N. Sumikawa, and L. Winemberg, "Learning the process for correlation analysis," *IEEE VLSI Test Symposium*, 2017.
- [5] K.-K. Hsieh, L.-C. Wang, W. Chen, and J. Bhadra, "Learning to produce direct tests for security verification using constrained process discovery," *ACM/IEEE Design Automation Conference*, pp. 1–6, 2017.
- [6] L.-C. Wang, "Llm-assisted analytics in semiconductor test (invited)," in *Proceedings of the 2024 ACM/IEEE International Symposium on Machine Learning for CAD*, ser. MLCAD '24. New York, NY, USA: Association for Computing Machinery, 2024. [Online]. Available: <https://doi.org/10.1145/3670474.3685974>
- [7] Google, "Dialogflow documentation," 2018. [Online]. Available: <https://cloud.google.com/dialogflow/docs>
- [8] T. B. B. et al., "Language models are few-shot learners," *CoRR (also in NeurIPS Proceedings)*, vol. abs/2005.14165, 2020. [Online]. Available: <https://arxiv.org/abs/2005.14165>
- [9] Y. Wang and et al., "Building a semantic parser overnight," in *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*. Association for Computational Linguistics, 2015.
- [10] J. Berant and P. Liang, "Semantic parsing via paraphrasing," in *Proceedings of the 52nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 2014.
- [11] R. Shin and et al., "Constrained language models yield few-shot semantic parsers," *CoRR*, vol. abs/2104.08768, 2021.
- [12] M. J. Yang, Y. J. Zeng, and L.-C. Wang, "Language driven analytics for failure pattern feedforward and feedback," in *IEEE International Test Conference*, 2022.
- [13] Y. J. Zeng, M. J. Yang, and L.-C. Wang, "Wafer map pattern analytics driven by natural language queries," in *IEEE International Test Conference in Asia*, 2022.
- [14] R. Wang, X. Han, L. Ji, S. Wang, T. Baldwin, and H. Li, "Toolgen: Unified tool retrieval and calling via generation," in *The Thirteenth International Conference on Learning Representations*, 2025. [Online]. Available: <https://openreview.net/forum?id=XLMMmowdY>
- [15] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu, Y. Lin, X. Cong, X. Tang, B. Qian, S. Zhao, L. Hong, R. Tian, R. Xie, J. Zhou, M. Gerstein, dahai li, Z. Liu, and M. Sun, "ToolLLM: Facilitating large language models to master 16000+ real-world APIs," in *The Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: <https://openreview.net/forum?id=dHng200Jjr>
- [16] Z. Wang, X. Zeng, W. Liu, L. Li, Y. Wang, L. Shang, X. Jiang, Q. Liu, and K.-F. Wong, "Toolflow: Boosting llm tool-calling through natural and coherent dialogue synthesis," 2025. [Online]. Available: <https://arxiv.org/abs/2410.18447>
- [17] H. Xu, Z. Wang, Z. Zhu, L. Pan, X. Chen, L. Chen, and K. Yu, "Alignment for efficient tool calling of large language models," 2025. [Online]. Available: <https://arxiv.org/abs/2503.06708>
- [18] H. Chase, "LangChain," Oct. 2022. [Online]. Available: <https://github.com/langchain-ai/langchain>
- [19] OpenAI, "GPT-3.5 Turbo," 2023. [Online]. Available: <https://platform.openai.com/docs/models>
- [20] M. Dupree, M. J. Yang, Y. J. Zeng, and L.-C. Wang, "IEA-Plot: Conducting wafer-based data analytics through chat," in *IEEE International Test Conference*. IEEE, 2023. [Online]. Available: https://youtu.be/_7X6vH6HaOw?si=hrxmpBzvM1x06Sk
- [21] W. Huang, P. Abbeel, D. Pathak, and I. Mordatch, "Language models as zero-shot planners: Extracting actionable knowledge for embodied agents," *CoRR*, vol. abs/2201.07207, 2022. [Online]. Available: <https://arxiv.org/abs/2201.07207>
- [22] M. A. et al., "Do as i can, not as i say: Grounding language in robotic affordances," *CoRR*, vol. abs/2204.01691, 2022. [Online]. Available: <https://arxiv.org/abs/2204.01691>
- [23] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," 2023. [Online]. Available: <https://arxiv.org/abs/2201.11903>
- [24] A. Plaat, A. Wong, S. Verberne, J. Broekens, N. van Stein, and T. Back, "Reasoning with large language models, a survey," 2024. [Online]. Available: <https://arxiv.org/abs/2407.11511>
- [25] OpenAI, "GPT-o3-mini," 2024. [Online]. Available: <https://platform.openai.com/docs/models>
- [26] LangChain, "LangGraph," 2024. [Online]. Available: <https://github.com/langchain-ai/langgraph>
- [27] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *ICLR*, 2023. [Online]. Available: https://openreview.net/forum?id=WE_vluYUL-X
- [28] N. Shinn, F. Cassano, A. Gopinath, K. R. Narasimhan, and S. Yao, "Reflexion: language agents with verbal reinforcement learning," in *Thirty-seventh Conference on Neural Information Processing Systems*, 2023. [Online]. Available: <https://openreview.net/forum?id=vAElhFcKW6>
- [29] S. Kim, Y. Su, and L.-C. Wang, "IEA-Plugin: An AI Agent Reasoner for Test Data Analytics," 2025. [Online]. Available: <https://arxiv.org/abs/2504.11496>
- [30] Q. Dong, J. Xu, L. Kong, Z. Sui, and L. Li, "Statistical knowledge assessment for large language models," 2023. [Online]. Available: <https://arxiv.org/abs/2305.10519>
- [31] W. Wang, B. Haddow, A. Birch, and W. Peng, "Assessing the reliability of large language model knowledge," 2023. [Online]. Available: <https://arxiv.org/abs/2310.09820>
- [32] L. Luo, T.-T. Vu, D. Phung, and G. Haffari, "Systematic assessment of factual knowledge in large language models," 2023. [Online]. Available: <https://arxiv.org/abs/2310.11638>
- [33] X. Yin, X. Zhang, J. Ruan, and X. Wan, "Benchmarking knowledge boundary for large language models: A different perspective on model evaluation," 2024. [Online]. Available: <https://arxiv.org/abs/2402.11493>
- [34] F. Bianchini, "Evaluating intelligence and knowledge in large language models," *Topoi*, 2024. [Online]. Available: <https://api.semanticscholar.org/CorpusID:271661147>
- [35] X. Wang, J. Wei, D. Schuurmans, Q. V. Le, E. H. Chi, S. Narang, A. Chowdhery, and D. Zhou, "Self-consistency improves chain of thought reasoning in language models," in *ICLR 2023, Kigali, Rwanda, May 1-5, 2023*, 2023. [Online]. Available: <https://openreview.net/pdf?id=1PL1NIMMrw>
- [36] X. Chen, R. Aksitov, U. Alon, J. Ren, K. Xiao, P. Yin, S. Prakash, C. Sutton, X. Wang, and D. Zhou, "Universal self-consistency for large language models," in *ICML 2024 Workshop on In-Context Learning*, 2024. [Online]. Available: <https://openreview.net/forum?id=LjsjHF7nAN>
- [37] J. Zhuo, S. Zhang, X. Fang, H. Duan, D. Lin, and K. Chen, "ProSA: Assessing and understanding the prompt sensitivity of LLMs," in *EMNLP 2024*, Nov. 2024, pp. 1950–1976. [Online]. Available: <https://aclanthology.org/2024.findings-emnlp.108/>
- [38] M. Sclar, Y. Choi, Y. Tsvetkov, and A. Suhr, "Quantifying language models' sensitivity to spurious features in prompt design or: How i learned to start worrying about prompt formatting," in *ICLR 2024*, 2024. [Online]. Available: <https://openreview.net/forum?id=RIu5lyNXjT>
- [39] S. Lu, H. Schuff, and I. Gurevych, "How are prompts different in terms of sensitivity?" in *2024 NAACL: Human Language Technologies (Volume 1: Long Papers)*, Jun. 2024, pp. 5833–5856. [Online]. Available: <https://aclanthology.org/2024.naacl-long.325/>
- [40] Y. Chen, C. Zhao, Z. Yu, K. McKeown, and H. He, "On the relation between sensitivity and accuracy in in-context learning," in *EMNLP 2023*, Dec. 2023, pp. 155–167. [Online]. Available: <https://aclanthology.org/2023.findings-emnlp.12/>
- [41] Y. Xiang, H. Yan, L. Gui, and Y. He, "Addressing order sensitivity of in-context demonstration examples in causal language models," in *ACL 2024*, Aug. 2024, pp. 6467–6481. [Online]. Available: <https://aclanthology.org/2024.findings-acl.386/>
- [42] F. Errica, G. Siracusano, D. Sanvito, and R. Bifulco, "What did i do wrong? quantifying llms' sensitivity and consistency to prompt engineering," 2025. [Online]. Available: <https://arxiv.org/abs/2406.12334>
- [43] J. Liu, D. Shen, Y. Zhang, B. Dolan, L. Carin, and W. Chen, "What makes good in-context examples for gpt-3?" 2021. [Online]. Available: <https://arxiv.org/abs/2101.06804>